Tema 10

Colecciones de datos

Programación

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Tipos de colecciones (listas, pilas, colas, tablas) 5
2.1 Listas 5
2.2 Pilas 6
2.3 Colas 6
2.4 Tablas 7
3. Jerarquías de colecciones 9
4. Operaciones básicas con colecciones. Acceso a elementos y recorridos 12
4.1 Acceso a elementos 12
4.2 Recorridos 13
5. Uso de clases y métodos genéricos 15
5.1 Clases genéricas de colecciones - Phyton 15
5.2 Clases genéricas de colecciones - Java 19
5.3 Ventajas de las clases y métodos genéricos de colecciones 28
6. Resumen 29
Glosario 30
Bibliografía 31

Objetivos de la unidad

- Entender el concepto de colecciones de datos y su importancia en la programación.
- Identificar y utilizar diferentes tipos de colecciones de datos, como arrays, listas, diccionarios y conjuntos.
- Comprender cómo las colecciones de datos pueden ser utilizadas para resolver problemas específicos en la programación.
- Desarrollar habilidades para manipular, organizar y procesar datos utilizando colecciones de datos.
- Aprender las mejores prácticas para trabajar con colecciones de datos, incluyendo la gestión de errores y la optimización de la eficiencia.
- Familiarizarse con la manipulación de datos en tiempo real y en situaciones prácticas.
- Aplicar los conocimientos adquiridos en la unidad a proyectos y tareas de programación prácticos.
- Desarrollar habilidades de resolución de problemas y pensamiento crítico al trabajar con colecciones de datos en la programación.

1. Introducción

Las colecciones de datos son estructuras que nos permiten almacenar y organizar grandes cantidades de información de manera eficiente. En la programación, estas estructuras son esenciales para realizar tareas específicas y resolver problemas complejos.

En esta unidad, aprenderás sobre diferentes tipos de colecciones de datos, como arrays, listas, diccionarios y conjuntos, y cómo utilizarlos para manipular y procesar datos de manera efectiva. También exploraremos cómo podemos trabajar con colecciones de datos para resolver problemas prácticos y hacer nuestros programas más eficientes.

Además, a medida que avancemos en la unidad, discutiremos las mejores prácticas para trabajar con colecciones de datos, incluyendo la organización, la gestión de errores y la manipulación de datos en tiempo real.

Tech Hub <FP> euroformac

2. Tipos de colecciones (listas, pilas, colas, tablas)

2.1 Listas

Una **lista** es una colección ordenada de elementos que pueden ser del mismo tipo o de tipos diferentes. Cada elemento de la lista tiene una **posición única** dentro de la misma, lo que permite acceder a él mediante su **índice**.

En Python, las listas se definen con **corchetes** []. Por ejemplo, mi_lista = [1, 2, 3, 4, 5]. Los elementos pueden ser añadidos o eliminados mediante diversos métodos.

```
mi_lista = [1, 2, 3, 4, 5]  # Creación de una lista
print(mi_lista)  # [1, 2, 3, 4, 5]

# Añadir elementos a la lista
mi_lista.append(6)
print(mi_lista)  # [1, 2, 3, 4, 5, 6]

# Eliminar elementos de la lista
mi_lista.remove(2)
print(mi_lista)  # [1, 3, 4, 5, 6]
```

Más info

Con este vídeo aprenderás a usar las listas en Python: https://www.youtube.com/watch?v=VsSMC1A7_F0&ab_channel=Dimas

Importante

En Python, las listas son más eficientes que los arrays debido a que ofrecen más funcionalidades, como agregar y eliminar elementos de forma dinámica.

2.2 Pilas

Una **pila** es una colección de elementos donde el último en entrar es el primero en salir (**LIFO**, por sus siglas en inglés). Esto significa que el último elemento que se añade a la pila es el primero que se elimina.

Las pilas se utilizan a menudo para llevar un registro de las **llamadas** a funciones, donde cada llamada se añade a la pila y se elimina cuando la función devuelve un valor.

En Python, las pilas se pueden implementar mediante una lista utilizando los métodos append() para añadir elementos y pop() para eliminarlos.

```
mi_pila = []  # Creación de una pila vacía


# Añadir elementos a la pila
mi_pila.append(1)
mi_pila.append(2)
mi_pila.append(3)
print(mi_pila)  # [1, 2, 3]


# Eliminar elementos de la pila
mi_pila.pop()
print(mi_pila)  # [1, 2]
```

2.3 Colas

A diferencia de las pilas, las **colas** (**FIFO**, por sus siglas en inglés) tienen un comportamiento en el que el primer elemento que se añade a la cola es el primero en salir.

Las colas se utilizan a menudo en algoritmos de búsqueda, donde los elementos que se encuentran por primera vez en la cola son los primeros que se examinan.

En Python, las colas se pueden implementar utilizando la biblioteca queue.

```
from queue import Queue
```

```
mi_cola = Queue()  # Creación de una cola vacía

# Añadir elementos a la cola
mi_cola.put(1)
mi_cola.put(2)
mi_cola.put(3)
print(mi_cola.queue)  # [1, 2, 3]

# Eliminar elementos de la cola
mi_cola.get()
print(mi_cola.queue)  # [2, 3]
```

Más info

En este vídeo verás una explicación teórica y práctica sobre la pila y la cola.

https://www.youtube.com/watch?v=ceXAL2_nhV0&ab_channel=Sklyvan

2.4 Tablas

Una **tabla** es una colección de elementos que se organizan en **filas** y **columnas**. Las tablas se utilizan comúnmente para almacenar datos estructurados.

En Python, las tablas se pueden implementar utilizando listas anidadas, donde cada fila se representa como una lista de elementos y la tabla es una lista de filas.

```
mi_tabla = [
    ['Nombre', 'Edad', 'Ciudad'],
    ['Juan', 20, 'Madrid'],
```

```
    ['María', 25, 'Barcelona'],
    ['Pedro', 30, 'Valencia']
]

# Acceder a los elementos de la tabla
print(mi_tabla[1][0])  # Juan
print(mi_tabla[2][2])  # Barcelona

# Añadir filas a la tabla
mi_tabla.append(['Ana', 22, 'Sevilla'])
print(mi_tabla)
```

Importante

Las colecciones de datos son una parte fundamental de la programación en Java y Python, y son esenciales para el manejo y la manipulación de grandes cantidades de información.

3. Jerarquías de colecciones

Las jerarquías de colecciones se refieren a la organización de diferentes tipos de estructuras de datos en un sistema jerárquico en el que los tipos de datos se clasifican en diferentes niveles basados en su complejidad y sus capacidades.

La jerarquía de colecciones se compone de varios niveles, que incluyen:

- **Colecciones atómicas:** Son los tipos de datos más básicos y simples, que no pueden subdividirse en partes más pequeñas. Ejemplos de colecciones atómicas incluyen números, caracteres y cadenas de caracteres.
- **Colecciones homogéneas:** Son colecciones de elementos del mismo tipo de datos. Las colecciones homogéneas incluyen listas, tuplas y arreglos.
- **Colecciones heterogéneas:** Son colecciones de elementos de diferentes tipos de datos. Las colecciones heterogéneas incluyen diccionarios y conjuntos.
- **Colecciones complejas:** Son colecciones que están formadas por diferentes tipos de colecciones. Ejemplos de colecciones complejas incluyen estructuras de datos en árbol, gráficos y bases de datos.

Es importante tener en cuenta que cada nivel en la jerarquía de colecciones puede incluir todos los tipos de colecciones inferiores, y a su vez, cada tipo de colección puede incluir todas las colecciones de nivel inferior en su interior.

Aquí tienes algunos ejemplos de jerarquía de colecciones en Python:

- **Colecciones atómicas:** int, float, str, bool.

```
# Colecciones atómicas:
x = 5          # entero
y = 3.14       # flotante
z = "Hola"     # cadena de caracteres
w = True       # booleano
```

- **Colecciones homogéneas:** list, tuple, array.

```
# Lista:
mi_lista = [1, 2, 3, 4, 5]
```

- **Colecciones heterogéneas:** dict, set.

```
# Tupla:
mi_tupla = (1, 2, 3, 4, 5)
```

- **Colecciones complejas:** árboles, gráficos, bases de datos.
  - **Árboles**

```
# Arbol:
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izquierdo = None
        self.derecho = None

raiz = Nodo(1)
raiz.izquierdo = Nodo(2)
raiz.derecho = Nodo(3)
raiz.izquierdo.izquierdo = Nodo(4)
raiz.izquierdo.derecho = Nodo(5)
```

  - **Gráficos**

```
# Grafo:
grafo = {
    'A': ['B', 'C'],
    'B': ['C', 'D'],
    'C': ['D'],
    'D': ['C'],
    'E': ['F'],
    'F': ['C']
}
```

- **Bases de datos**

```
# Base de datos (utilizando la biblioteca SQLite):

import sqlite3

conexion = sqlite3.connect('mi_base_de_datos.db')
cursor = conexion.cursor()

# Crear una tabla
cursor.execute('''CREATE TABLE alumnos (nombre TEXT, edad INTEGER)''')

# Insertar datos en la tabla
cursor.execute("INSERT INTO alumnos VALUES ('Juan', 25)")

# Consultar datos de la tabla
cursor.execute("SELECT * FROM alumnos")
registros = cursor.fetchall()
print(registros)

conexion.commit()
conexion.close()
```

4. Operaciones básicas con colecciones. Acceso a elementos y recorridos

Las operaciones básicas que se pueden realizar con cualquier colección son:

- Creación de la colección.
- Inserción de elementos.
- Eliminación de elementos.
- Modificación de elementos.
- Búsqueda de elementos.

En Python, las operaciones básicas varían según el tipo de colección. Por ejemplo, para las listas se pueden utilizar las siguientes operaciones:

- Creación de la lista: mi_lista = [1, 2, 3]
- Inserción de elementos: mi_lista.append(4) o mi_lista.insert(2, 5)
- Eliminación de elementos: mi_lista.remove(3) o del mi_lista[1]
- Modificación de elementos: mi_lista[0] = 0
- Búsqueda de elementos: indice = mi_lista.index(2)

Para los diccionarios, las operaciones básicas son un poco diferentes:

- Creación del diccionario: mi_diccionario = {"nombre": "Juan", "edad": 25}
- Inserción de elementos: mi_diccionario["altura"] = 1.75
- Eliminación de elementos: del mi_diccionario["edad"]
- Modificación de elementos: mi_diccionario["nombre"] = "Pedro"
- Búsqueda de elementos: valor = mi_diccionario.get("nombre", "No encontrado")

4.1 Acceso a elementos

El acceso a los elementos de una colección se realiza a través de su índice o clave. En Python, los índices comienzan en 0 para la mayoría de las colecciones.

Por ejemplo, para acceder al segundo elemento de una lista se utiliza mi_lista[1]. Para acceder al valor correspondiente a una clave en un diccionario se utiliza mi_diccionario["nombre"].

4.2 Recorridos

Hay diferentes formas de recorrer los elementos de una colección en Python, dependiendo del tipo de colección y del objetivo del recorrido. A continuación, se presentan algunos de los tipos de recorridos más comunes:

- Recorrido con for: Se utiliza para recorrer todos los elementos de una colección.
  Por ejemplo:

```
mi_lista = [1, 2, 3]
for elemento in mi_lista:
    print(elemento)
```

- Recorrido con while: Se utiliza cuando se quiere recorrer la colección hasta que se cumpla una determinada condición. Por ejemplo:

```
mi_lista = [1, 2, 3]
i = 0
while i < len(mi_lista):
    print(mi_lista[i])
    i += 1
```

- Recorrido inverso con reversed: Se utiliza para recorrer los elementos de una colección en orden inverso. Por ejemplo:

```
mi_lista = [1, 2, 3]
for elemento in reversed(mi_lista):
    print(elemento)
```

- Recorrido con índices: Se utiliza cuando es necesario acceder a los índices de los elementos de una colección. Por ejemplo:

```
mi_lista = ['a', 'b', 'c', 'd', 'e']
for i in range(len(mi_lista)):
    print("El índice {} contiene el valor {}".format(i, mi_lista[i]))
```

En este ejemplo, utilizamos la función range para crear una secuencia de números del 0 al 4 (que son los índices de los elementos en la lista mi_lista). Luego, recorremos esta secuencia y utilizamos cada número como índice para acceder al elemento correspondiente en la lista y mostrarlo en la salida.

El resultado sería:

```
El índice 0 contiene el valor a
El índice 1 contiene el valor b
El índice 2 contiene el valor c
El índice 3 contiene el valor d
El índice 4 contiene el valor e
```

5. Uso de clases y métodos genéricos

En la programación, a menudo necesitamos trabajar con colecciones de datos, como listas, conjuntos o mapas. En lugar de escribir clases y métodos separados para cada tipo de colección de datos, podemos utilizar clases y métodos genéricos para trabajar con cualquier tipo de colección de datos.

Definición

Una colección de datos es una estructura de datos que permite almacenar y manipular múltiples valores.

5.1 Clases genéricas de colecciones - Phyton

En Python, las clases se pueden utilizar para trabajar con colecciones de datos de varias maneras. Aquí se deja un esquema con todas las posibles clases de colecciones de datos:

Clase en Phyton	Métodos	Descripción
list	Append() Remove() Len()	Una secuencia mutable de elementos
tuple	Index() Count() Len()	Una secuencia inmutable de elementos
set	Add() Remove() Len()	Un conjunto no ordenado de elementos
frozenset	Isdisjoint() Issubset() Issuperset() Len()	Un conjunto inmutable no ordenado de elementos
dict	Keys() Values() Ítems() Get()	Un diccionario mutable de pares clave-valor

defaultdict	Defaultdict() Keys() Values() Ítems()	Un diccionario mutable de pares clave-valor con un valor predeterminado para claves inexistentes
OrderedDict	Move_to_end() Popitem() Keys() Values() Ítems()	Un diccionario mutable ordenado de pares clave-valor
Counter	Elements() Most_common() Subtract() Update()	Un diccionario mutable de conteo de elementos
deque	Append() Pop() Appendleft() Popleft() Rotate() Reverse() Extend() Extendleft()	Una cola doblemente terminada mutable

A continuación, se presentan algunas formas en las que se pueden utilizar las clases para trabajar con colecciones de datos:

- Crear una clase personalizada para la colección de datos: Puedes crear una clase personalizada que represente una colección de datos específica, como una lista, una pila, una cola, un diccionario, etc. En esta clase, puedes definir métodos para agregar, eliminar, buscar y actualizar elementos en la colección. También puedes definir propiedades y métodos adicionales para realizar operaciones específicas con la colección.

  Por ejemplo, aquí hay un ejemplo de una clase personalizada que representa una lista de números:

```
class NumberList:
    def __init__(self):
        self._numbers = []
```

```
    def add_number(self, number):
        self._numbers.append(number)

    def remove_number(self, number):
        self._numbers.remove(number)

    def get_numbers(self):
        return self._numbers

    def get_sum(self):
        return sum(self._numbers)
```

En este ejemplo, la clase NumberList tiene métodos para agregar, eliminar y obtener los números en la lista, así como un método adicional para obtener la suma de los números.

- Crear una subclase de una colección existente: Puedes crear una subclase de una colección existente, como la clase list de Python, y agregar propiedades y métodos adicionales a la subclase.

  Por ejemplo, aquí hay un ejemplo de una subclase personalizada de la clase **list** de Python:

```
class NumberList(list):
    def get_sum(self):
        return sum(self)
```

En este ejemplo, la clase NumberList hereda de la clase list de Python y agrega un método adicional para obtener la suma de los números en la lista.

Utilizar una clase existente de la biblioteca estándar de Python: Python proporciona varias clases de colecciones de datos en su biblioteca estándar, como list, tuple, dict, set, etc. Puedes utilizar estas clases directamente o como parte de una clase personalizada o subclase.

Aquí hay un ejemplo de cómo utilizar la clase dict de Python para almacenar pares clave-valor:

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
print(my_dict['a'])  # Output: 1
my_dict['d'] = 4
print(my_dict)  # Output: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

En este ejemplo, se utiliza la clase dict de Python para almacenar pares clave-valor, y se utilizan métodos como __getitem__ y __setitem__ para acceder y actualizar los elementos en el diccionario.

Utilizar la biblioteca estándar collections: Python proporciona la biblioteca estándar collections que ofrece varias clases de colecciones de datos especializadas, como deque, OrderedDict, Counter, defaultdict, entre otras. Estas clases son útiles cuando necesitas una colección con ciertas características específicas.

Aquí hay un ejemplo de cómo utilizar la clase deque de collections para crear una cola de elementos:

```
from collections import deque

my_queue = deque([1, 2, 3])
my_queue.append(4)
print(my_queue.popleft())  # Output: 1
print(my_queue)  # Output: deque([2, 3, 4])
```

En este ejemplo, se utiliza la clase deque de collections para crear una cola de elementos y se utilizan métodos como append y popleft para agregar y eliminar elementos en la cola.

Utilizar la biblioteca de terceros numpy: Si trabajas con grandes colecciones de datos numéricos, la biblioteca numpy es una excelente opción. numpy proporciona una clase de arreglo multidimensional (ndarray) que es muy eficiente para realizar operaciones numéricas.

Aquí hay un ejemplo de cómo utilizar la clase ndarray de numpy para crear una matriz y realizar algunas operaciones:

```
import numpy as np


my_array = np.array([[1, 2], [3, 4]])
print(my_array)  # Output: array([[1, 2], [3, 4]])
print(my_array.transpose())  # Output: array([[1, 3], [2, 4]])
print(my_array.flatten())  # Output: array([1, 2, 3, 4])
```

En este ejemplo, se utiliza la clase ndarray de numpy para crear una matriz y se utilizan métodos como transpose y flatten para realizar operaciones en la matriz.

En resumen, las clases son una herramienta poderosa para trabajar con colecciones de datos en Python. Ya sea que necesites una colección personalizada con características específicas o una clase de colección de datos existente de la biblioteca estándar o de terceros, Python tiene muchas opciones para elegir.

Sabías que

Las clases genéricas de colecciones de datos en Python incluyen: list, tuple, set, frozenset, dict, defaultdict, OrderedDict, Counter, deque y más.

5.2 Clases genéricas de colecciones - Java

En Java, también es posible utilizar clases para trabajar con colecciones de datos. Java ofrece una amplia gama de clases y interfaces de colecciones que se pueden utilizar para almacenar y manipular conjuntos de datos. A continuación, se presenta un cuadro resumen con las clases de colecciones de datos:

Interface	Methods	Description	Implementing Classes
Collection<E>	Add(E) Remove(Object) Size()	The root interface in the collection hierarchy	List, set, queue, deque, etc
List<E>	Add(index, element) Remove (index) Get(index)	An ordered collection (also know as a sequence)	Arraylist, Linkedlist, vector, etc
Set<E>	Add(E) Remove(object) Contains(object)	A collection that contains no duplicate elements	Hashset, treeset, linkedhashset,etc
Queue<E>	Offer(E) Poll() Peek()	A collection designed for holding elements prior to processing	Linkedlist, priorityqueue, etc
Deque<E>	AddFirst(E) AddLast(E) RemoveFirst() RemoveLast()	A double-ended queue	Linkedlist, arraydeque, etc
Map<K, V>	Put(K, V) Get(object) Remove(object)	An object that maps keys to values	HasMap, TreeMap, LinkedHashMap, etc

Aquí vemos algunos ejemplos:

- **Utilizar la interfaz List:** La interfaz List se utiliza para almacenar una secuencia ordenada de elementos. Existen varias clases que implementan esta interfaz, como ArrayList, LinkedList, Vector, entre otras. La elección de una clase dependerá del tipo de operaciones que se quieran realizar y de la eficiencia que se necesite.
  Aquí hay un ejemplo de cómo utilizar la clase ArrayList para crear una lista de números y realizar algunas operaciones:

```
import java.util.ArrayList;
```

```
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> myNumbers = new ArrayList<Integer>();
        myNumbers.add(10);
        myNumbers.add(15);
        myNumbers.add(20);
        System.out.println(myNumbers); // Output: [10, 15, 20]
        myNumbers.remove(1);
        System.out.println(myNumbers); // Output: [10, 20]
    }
}
```

Importante

En Java, la clase ArrayList es más eficiente que la clase Vector debido a que no es sincronizada, lo que significa que no tiene los costos de sincronización adicionales que se producen en la clase Vector.

En este ejemplo, se utiliza la clase ArrayList para crear una lista de números y se utilizan métodos como add y remove para agregar y eliminar elementos de la lista.

- **Utilizar la interfaz Set:** La interfaz Set se utiliza para almacenar una colección de elementos únicos. No permite elementos duplicados y su orden no está definido. Al igual que List, existen varias clases que implementan esta interfaz, como HashSet, LinkedHashSet, TreeSet, entre otras.
  Aquí hay un ejemplo de cómo utilizar la clase HashSet para crear un conjunto de nombres y realizar algunas operaciones:

```
import java.util.HashSet;


public class Main {
```

```
    public static void main(String[] args) {
        HashSet<String> myNames = new HashSet<String>();
        myNames.add("John");
        myNames.add("Sarah");
        myNames.add("Mike");
        myNames.add("John");
        System.out.println(myNames); // Output: [John, Sarah, Mike]
        myNames.remove("Mike");
        System.out.println(myNames); // Output: [John, Sarah]
    }
}
```

En este ejemplo, se utiliza la clase HashSet para crear un conjunto de nombres y se utilizan métodos como add y remove para agregar y eliminar elementos del conjunto.

- **Utilizar la interfaz Map:** La interfaz Map se utiliza para almacenar una colección de pares clave-valor. Cada elemento del mapa tiene una clave única y un valor asociado. Al igual que List y Set, existen varias clases que implementan esta interfaz, como HashMap, LinkedHashMap, TreeMap, entre otras.
  Aquí hay un ejemplo de cómo utilizar la clase HashMap para crear un mapa de nombres y edades y realizar algunas operaciones:

```
import java.util.HashMap;


public class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> myMap = new HashMap<String, Integer>();
        myMap.put("John", 25);
        myMap.put("Sarah", 30);
        myMap.put("Mike", 20);
        System.out.println(myMap);   //   Output:   {John=25,   Mike=20,
Sarah=30}
        myMap.remove("Mike");
```

```
        System.out.println(myMap); // Output: {John=25, Sarah=30}
    }
}
```

En este ejemplo, se utiliza la clase HashMap para crear un mapa de nombres y edades y se utilizan métodos como put y remove para agregar y eliminar elementos del mapa.

Estos son solo algunos ejemplos de cómo se pueden utilizar las clases de colecciones en Java. Hay muchas más opciones disponibles, dependiendo del tipo de datos que se quieran almacenar y de las operaciones que se quieran realizar. En general, las clases de colecciones en Java son muy poderosas y pueden ser muy útiles para trabajar con grandes conjuntos de datos.

- **Utilizar la interfaz Queue**: La interfaz Queue se utiliza para almacenar una colección de elementos en la que el elemento que se agrega primero es el que se elimina primero (también conocida como estructura de datos **FIFO**, por sus siglas en inglés). Existen varias clases que implementan esta interfaz, como LinkedList, ArrayDeque, entre otras.
  Aquí hay un ejemplo de cómo utilizar la clase LinkedList para crear una cola de nombres y realizar algunas operaciones:

```
import java.util.LinkedList;
import java.util.Queue;

public class Main {
    public static void main(String[] args) {
        Queue<String> myQueue = new LinkedList<String>();
        myQueue.add("John");
        myQueue.add("Sarah");
        myQueue.add("Mike");
        System.out.println(myQueue); // Output: [John, Sarah, Mike]
        myQueue.remove();
        System.out.println(myQueue); // Output: [Sarah, Mike]
    }
}
```

En este ejemplo, se utiliza la clase LinkedList para crear una cola de nombres y se utilizan métodos como add y remove para agregar y eliminar elementos de la cola.

- **Utilizar la interfaz Deque:** La interfaz Deque se utiliza para almacenar una colección de elementos en la que se puede agregar y eliminar elementos tanto al principio como al final de la colección (también conocida como estructura de datos doblemente enlazada). Existen varias clases que implementan esta interfaz, como LinkedList, ArrayDeque, entre otras.

  Aquí hay un ejemplo de cómo utilizar la clase LinkedList para crear una deque de números y realizar algunas operaciones:

```
import java.util.LinkedList;
import java.util.Deque;

public class Main {
    public static void main(String[] args) {
        Deque<Integer> myDeque = new LinkedList<Integer>();
        myDeque.addFirst(10);
        myDeque.addLast(20);
        myDeque.addFirst(5);
        System.out.println(myDeque); // Output: [5, 10, 20]
        myDeque.removeLast();
        System.out.println(myDeque); // Output: [5, 10]
    }
}
```

En este ejemplo, se utiliza la clase LinkedList para crear una deque de números y se utilizan métodos como addFirst y addLast para agregar elementos al principio y al final de la deque, respectivamente. También se utiliza el método removeLast para eliminar el último elemento de la deque.

- **Utilizar la clase Arrays:** La clase Arrays proporciona varios métodos estáticos para trabajar con matrices, como sort, binarySearch, equals, entre otros.

Aquí hay un ejemplo de cómo utilizar el método binarySearch de la clase Arrays para buscar un número en una matriz:

```
import java.util.Arrays;


public class Main {
    public static void main(String[] args) {
        int[] myArray = {10, 20, 30, 40, 50};
        int index = Arrays.binarySearch(myArray, 30);
        System.out.println("Index of 30: " + index); // Output: Index of
30: 2
    }
}
```

En este ejemplo, se utiliza el método binarySearch de la clase Arrays para buscar el número 30 en una matriz de números. El método devuelve el índice del elemento buscado si lo encuentra, o un valor negativo si no lo encuentra.

- **Utilizar la clase Collections:** La clase Collections proporciona varios métodos estáticos para trabajar con colecciones, como sort, binarySearch, max, min, entre otros.
  Aquí hay un ejemplo de cómo utilizar el método sort de la clase Collections para ordenar una lista de números:

```
import java.util.ArrayList;
import java.util.Collections;


public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> myList = new ArrayList<Integer>();
        myList.add(10);
        myList.add(5);
        myList.add(20);
        Collections.sort(myList);
        System.out.println(myList); // Output: [5, 10, 20]
```

```
    }
}
```

En este ejemplo, se utiliza el método sort de la clase Collections para ordenar una lista de números en orden ascendente.

- **Utilizar la clase LinkedList:** La clase LinkedList es una lista doblemente enlazada que implementa la interfaz List. La principal ventaja de usar una LinkedList es que se pueden agregar y eliminar elementos de manera más eficiente que con un ArrayList, especialmente si se trata de grandes cantidades de elementos. Aquí hay un ejemplo:

```
import java.util.LinkedList;


public class Main {
    public static void main(String[] args) {
        LinkedList<String> myList = new LinkedList<String>();
        myList.add("elemento1");
        myList.add("elemento2");
        myList.add("elemento3");
        System.out.println(myList); // Output: [elemento1, elemento2,
elemento3]
    }
}
```

En este ejemplo, se crea una LinkedList de cadenas y se agregan algunos elementos a la lista. La LinkedList también tiene métodos para agregar y eliminar elementos de la cabeza o la cola de la lista.

- **Utilizar la clase HashSet:** La clase HashSet es una colección que no permite duplicados y no mantiene el orden de inserción de los elementos. Aquí hay un ejemplo:

```
import java.util.HashSet;


public class Main {
    public static void main(String[] args) {
        HashSet<Integer> mySet = new HashSet<Integer>();
        mySet.add(10);
```

```
        mySet.add(5);
        mySet.add(20);
        System.out.println(mySet); // Output: [5, 10, 20]
    }
}
```

En este ejemplo, se crea un HashSet de enteros y se agregan algunos elementos a la colección. El HashSet no mantiene el orden de inserción de los elementos.

- **Utilizar la clase TreeSet:** La clase TreeSet es una colección que mantiene los elementos ordenados en función de su valor. Aquí hay un ejemplo:

```
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> mySet = new TreeSet<Integer>();
        mySet.add(10);
        mySet.add(5);
        mySet.add(20);
        System.out.println(mySet); // Output: [5, 10, 20]
    }
```

En este ejemplo, se crea un TreeSet de enteros y se agregan algunos elementos a la colección. El TreeSet mantiene los elementos ordenados en función de su valor.

Sabías que

Las clases genéricas de colecciones de datos en Java incluyen: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap y más.

5.3 Ventajas de las clases y métodos genéricos de colecciones

Las clases y métodos genéricos de colecciones tienen varias ventajas:

- **Reutilización de código:** Las clases y métodos genéricos de colecciones nos permiten escribir código que se puede reutilizar con diferentes tipos de colecciones de datos.

- **Flexibilidad:** Las clases y métodos genéricos de colecciones nos permiten trabajar con diferentes tipos de colecciones de datos sin tener que escribir código separado para cada tipo de colección de datos.

- **Seguridad de tipos:** Las clases y métodos genéricos de colecciones nos permiten especificar el tipo de datos.

6. Resumen

En esta unidad has aprendido sobre los diferentes tipos de colecciones que existen en Java, como las listas, pilas, colas y tablas, y su jerarquía de herencia. Además, has visto las diferentes operaciones que se pueden realizar con estas colecciones, como el acceso a elementos y los recorridos, y has aprendido a utilizar las clases y métodos genéricos.

También has podido analizar el uso de algunas de las implementaciones más comunes, como ArrayList, LinkedList y HashMap, y su rendimiento en diferentes escenarios. Has adquirido conocimientos esenciales para manejar colecciones de datos de manera eficiente en Java, lo que te permitirá trabajar con mayor eficacia en proyectos de programación.

Es importante destacar la utilidad de las colecciones de datos en cualquier aplicación que maneje grandes cantidades de información y la necesidad de conocerlas a fondo para aprovechar todas sus posibilidades.

Tech Hub <FP> euroformac

Glosario

- **Clases y métodos genéricos:** Clases y métodos que pueden trabajar con diferentes tipos de datos, lo que aumenta la reutilización de código y la flexibilidad de las colecciones.

- **Cola:** Una colección de elementos que sigue la regla FIFO (primero en entrar, primero en salir) para agregar y eliminar elementos.

- **Colección:** Un objeto que agrupa múltiples elementos de datos en una sola unidad.

- **Jerarquía de colecciones:** Estructura de herencia de clases en la que se basan las colecciones, donde las interfaces son la capa más alta y las clases concretas son la capa más baja.

- **Lista:** Una colección ordenada de elementos que pueden tener duplicados y permiten el acceso a través de un índice.

- **Operaciones con colecciones:** Agregar, eliminar, buscar, ordenar y recorrer.

- **Pila:** Una colección de elementos que sigue la regla LIFO (último en entrar, primero en salir) para agregar y eliminar elementos.

- **Tabla:** Una colección de elementos que se organizan en filas y columnas, y se acceden a través de claves únicas.

Bibliografía

García, J. (2015). Estructuras de datos en Java. Madrid: Anaya Multimedia.

Rodríguez, M. (2019). Colecciones de datos en Python. Revista de Informática, 35(2), 45-56.

Sánchez, L. (2020). Tutorial sobre estructuras de datos en Java. Programación para todos. Recuperado de: https://www.programacionparatodos.com/tutorial-estructuras-datos-java/